

Unit 4

Microcontroller is a small and low cost microcomputer, which is used to perform the specific tasks of embedded systems like displaying microwave's info, receiving remote signal etc.

The general microcontroller consists of processor, memory (RAM, ROM, EPROM), serial ports, peripherals (timers, counters) etc.

microprocessor	microcontroller
- used for big Applications - designing & h/w cost is high - Not so easy to replace - Its power consumption is high because it has to control entire system - It can ^{doesn't} consist of CPU RAM, ROM, I/O ports - processing speed is above 1GHz	- used to execute a single task in an application - designing & h/w cost is low - easy to replace - It is built with CMOS tech which requires less power - It consists ^{consist} of RAM, ROM, I/O ports - Processing speed is about 8MHz to 50MHz

Types of microcontrollers

(i) 8-bit microcontroller

- used to execute arithmetic and logical operⁿ like addⁿ, subtraction, multiplication division etc.

ex Intel 8031 & 8051 are 8 bit microcontroller.

(ii) 16 bit

- used to perform Arithmetic & logical operⁿ where higher accuracy ~~reqd~~ & performance reqd.

ex 8096 .

(iii) 32 bit

- used in automatically controlled appliances like Automatic operational machines, medical appliances etc.

Applications of Microcontroller

- Lightsensing & controlling devices like LED
- Temp. sensing & controlling device like microwave oven
- fire detection & safety devices like Fire Alarm
- Measuring devices like volt meter

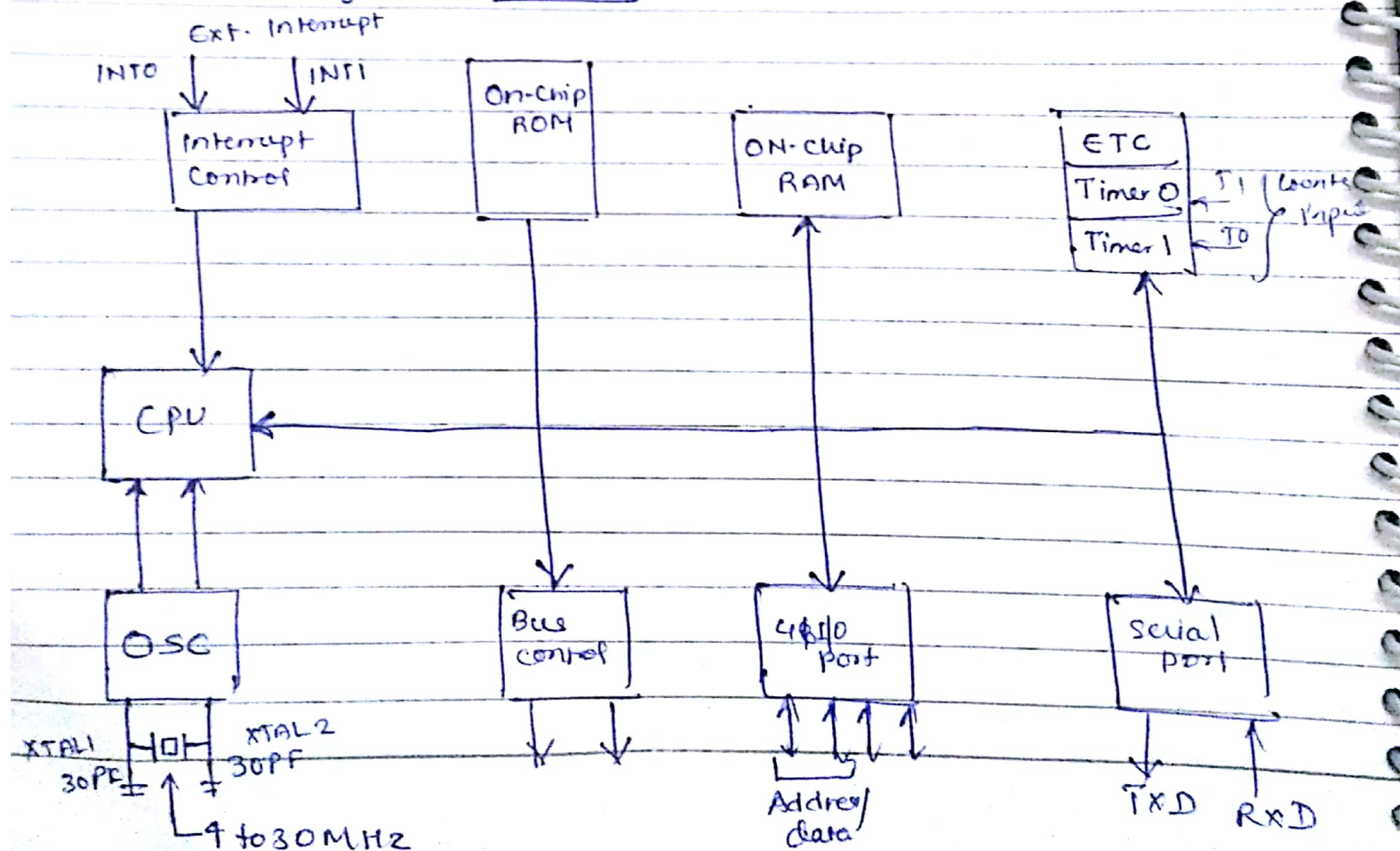
8051 microcontroller

It is designed by Intel in 1981. It is a 8-bit microcontroller.

Features

- (i) 8 bit CPU with Register A (accumulator) & B
- (ii) 16 bit program counter and Data pointer
- (iii) 8 bit PSW
- (iv) 8 bit stack pointer
- (v) Internal RAM of 128 bytes & Internal ROM (0 to 4K)
- (vi) 32 Input/output pins arranged as 4 8 bit ports
- (vii) Two 16 bit counters/timers
- (viii) full duplex serial data receiver/transmitter.
- (ix) Control register
- (x) Oscillator & clock circuit.

Block diagram of 8051



Components of 8051 Architecture

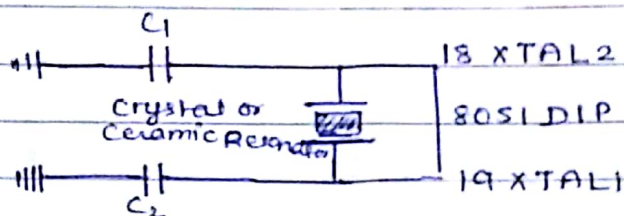
8051 Oscillator and Clock

A crystal is connected to form an oscillator. It is the heart of 8051 that generates the clock pulses by which all internal operations are synchronized.

Pin $XTAL_2$ & $XTAL_1$ are provided for connecting a resonant n/w to form an oscillator.

Typically a quartz crystal and capacitors are employed.

The crystal frequency is the basic internal clock frequency of the microcontroller.



Program Counter & Data pointer

Program Counter points to the address of next inst. to be executed

Data pointer (DPTR) consists of a high byte (DPH) and a low byte (DPL).

8051 contains 2-16 bit register PC, DPTR.

The PC is automatically incremented after every instruction byte is fetched and may also be altered by certain instruction.

PC is the only register that doesn't have internal address.

The DPTR register is made up of two 8-bit registers DPH & DPL, which are used to furnish memory addresses for internal & external code access and external data access.

8051 Flags and Program Status Word (PSW)

Flags are the flip flop which are used to store the result of program instruction.

PSW is used to define the flag.

Flags and the PSW

Flags are the flip flops which are used to store the results of program inst.

PSW used to define the flag.

Accumulator A & B — holds results of many instruction, particularly math & logical opern.

Internal memory

(i) Internal RAM

three ~~working~~ areas

(a) working register

(b) bit addressable

(c) General purpose RAM.

(ii) Internal ROM

Memory Organization in 8051

The 8051 microcontroller's memory is separated in Program Memory (ROM) and Data Memory (RAM).

The program memory i.e. ROM is used for storing the program to be executed i.e. instructions.

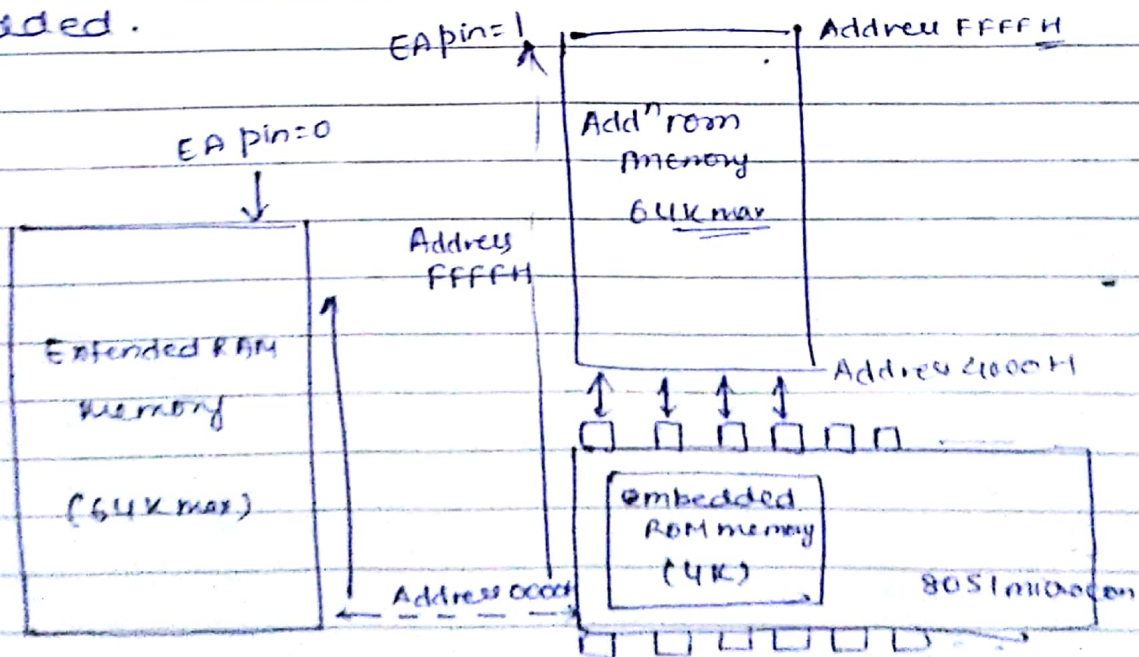
Data memory (RAM) is used for storing temporary variable data and intermediate results.

* Program Memory | Internal ROM of 8051 microcontroller

In 8051 microcontroller, the code or instruction to be executed are stored in the Internal ROM.

Original 8051 microcontroller has 4KB of Internal ROM.
i.e. occupies the code address space from 0000H to 0FFFH.

8051 memory org. allows external program memory to be added.



* Internal RAM | Data memory (or RAM memory Allocation in 8051)

It stores temporary data and intermediate results that generated and used during normal operation of microcontroller.

8051 has 128-byte Internal RAM.

the Internal RAM of 8051 is organised in 3 areas

- (i) Working registers
- (ii) bit addressable
- (iii) General purpose RAM

7FH	80	General purpose register
30H	16	Bit addressable registers
2FH		
20H		
1FH		
18H		
17H		
10H		
0FH		
08H		
07H		
00H		

(i) Working Register

- 32 bytes from address 00H to 1FH which make up to 32 working registers.
- Organized as 4 banks of 8 registers each
- Each register can be addressed by name or by its RAM address.
- Bit RSO & RSI in the PSW will determine which ~~bank~~ bank of registers is currently in use at any time
- Bank 0 is selected on Reset.

Working registers

Lower 128B (00-7FH)

(direct addressing and indirect)

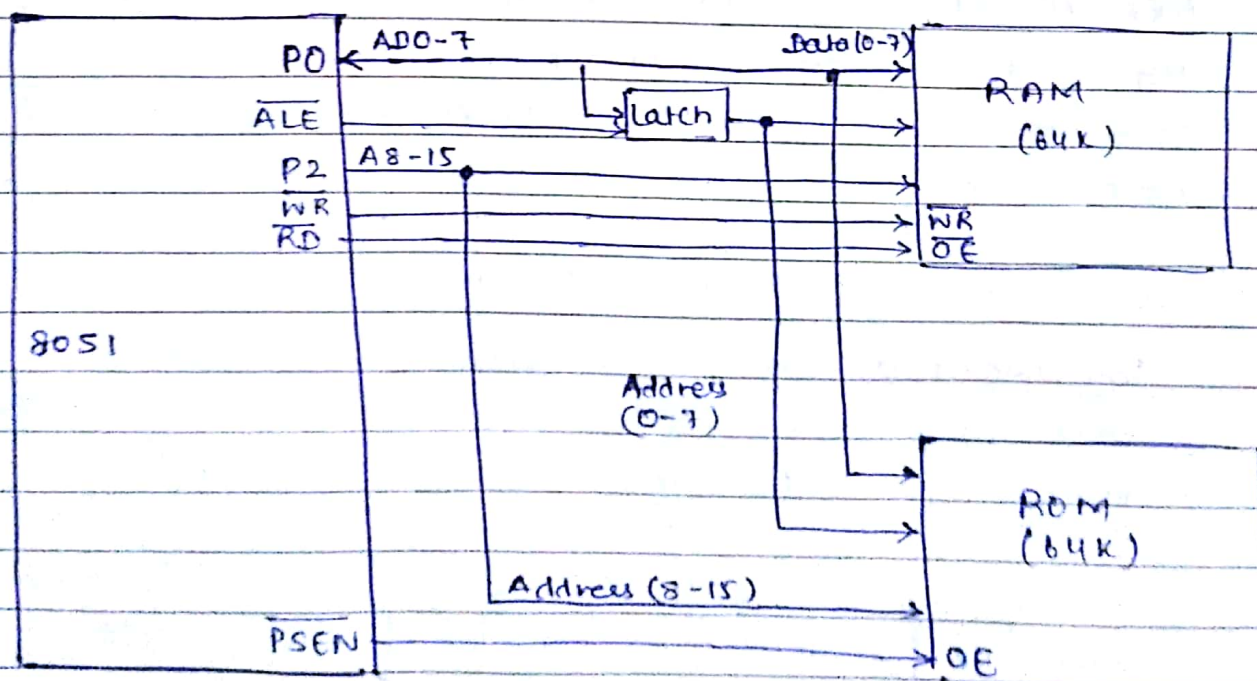
(ii) Bit addressable

- an area of 16 bytes occupies RAM byte address 20H to 2FH, forming a total 128 addressable bits
- These are used as useful only when the program need only to remember a binary event eg. Switch on light, switch off light etc.
- An addressable bit may be specified by its bit address of 00H to 7FH or 8 bits may form any byte address from 20H to 2FH.
- Each bit has its own address.

(iii) General purpose RAM

- area above bit area, from 30H to 7FH, addressable as bytes.

* Interfacing External Memory with 8051 microcontroller



SPECIAL FUNCTION REGISTER (SFR)

It acts as a control table that monitor and control the operation of 8051 microcontroller.

In Internal RAM structure, the Address space from 80H to FFH is allocated to SFRs.

Out of these 128 memory locations (80 to FFH) there are only 21 locations that are actually assigned to SFRs.

Each SFR has one byte address and also has a unique name which specifies its purpose.

Since SFRs are a part of Internal RAM structure, that we can access SFRs as if we access Internal RAM.

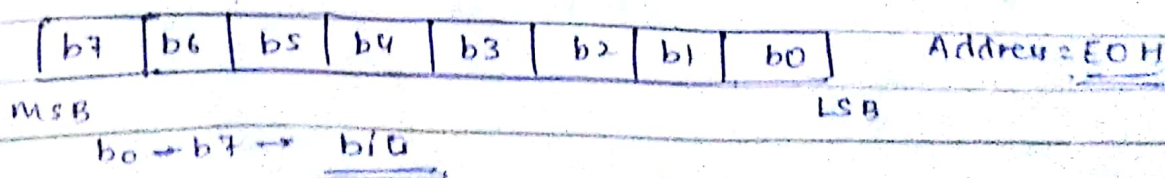
① Accumulator A and B

Accumulator A is the most important and most used by 8051 microcontroller SFRs.

Register A is located at the address E0H in SFR memory space.

It is used to hold data for almost all ALU operations.

The name 'accumulator' came from the fact that this register is used to accumulate (or store) the result of all Arithmetic & Logical operations.



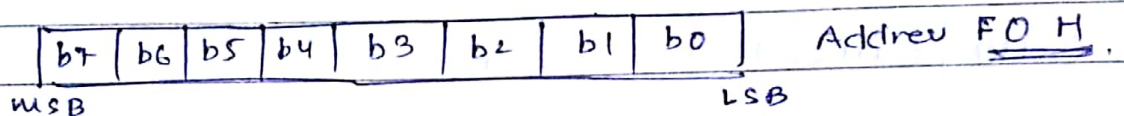
Register B

It is used along with ACC in multiplication & division operations.

These two operⁿ are performed on data that are stored only in Register A & B.

During multiplication, one of the operand (multiplier or multiplicand) is stored in B register and also the higher byte of register.

In case of division, it holds the divisor & remainder of ^{result}.



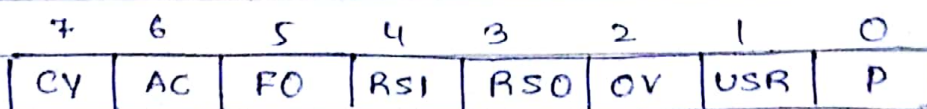
② Program Status Word (PSW)

It is also called as flag registers and is one of the important SFRs.

It consists of flag bits, which help the programmer in checking the condⁿ of result and also make decision.

Flags are 1-bit storage elements that store & indicate the nature of the result that is generated by execution of certain inst.

Address = DOH



CY - Carry flag: used in arithmetic, Jump, & Boolean inst.

AC - Auxiliary carry flag: Used for BCD arithmetic

FO - User Flag 0

RS1 - Register Bank select Bit 1

RS0 - Register Bank select Bit 0

} used in Internal RAM

RS1	RS0	BANK selected
0	0	→ BANK 0
0	1	→ BANK 1
1	0	→ BANK 2
1	1	→ BANK 3

OV - Overflow flag :- set by Arithmetic operⁿ

USB - User defined flag (Reserved)

P - Parity flag :- Shows parity of register A ; 1 = odd parity
otherwise reset.

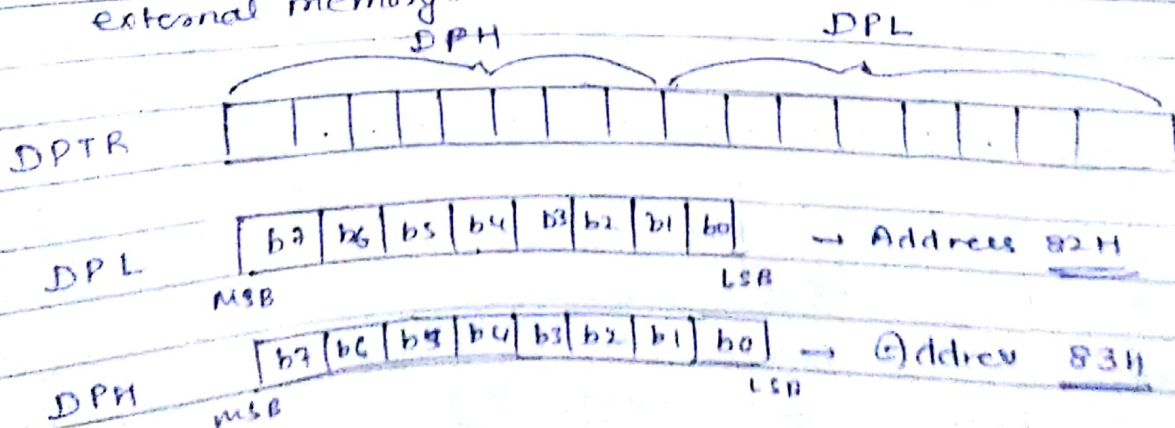
③ Data pointer Register (DPTR) → DPL & DPH

It is a 16 bit register and is physically the combⁿ of DPL (data pointer low) & DPH (data pointer High)

DPTR can be used as a single 16 bit register or two 8 bit DPL & DPH registers.

DPTR does not have a physical memory Address but DPL and DPH have separate address in SFR memory space i.e DPL = 82H & DPH = 83H.

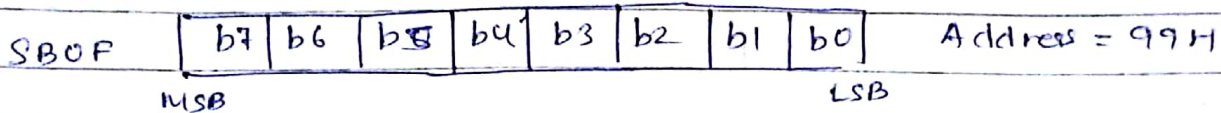
DPTR register is used by programmers addressing external memory.



* peripheral data Registers

⑥ SBUF (Serial data buffer)

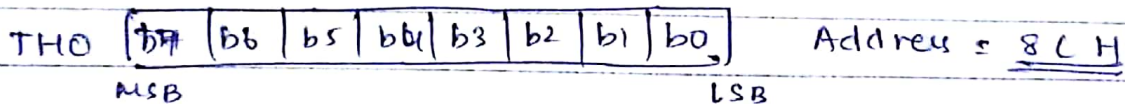
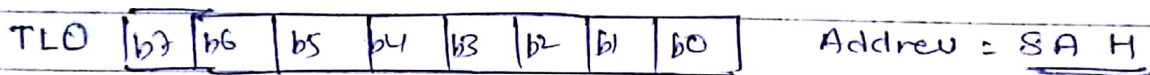
It is used to hold serial data while transmission or reception.



⑦ TLO | THO (Timer 0 Low/high)

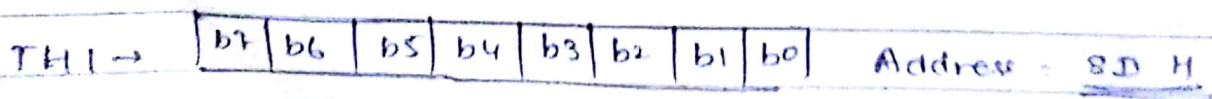
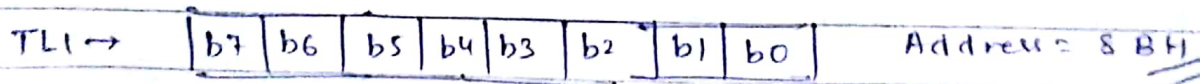
Timer 0 consists of two SFRs: TLO & THO.

TLO is the lower byte & THO is the higher byte together they form 16 bit Timer 0 register.



⑧ TL1 | TH1 (Timer 1 Low/high)

TL1 & TH1 are lower & higher bytes of timer 1



(Important)

* Peripheral Control Registers

⑨ TMOD (Timer Mode)

TMOD is used to set the operating modes of Timer 0 & T1.

The lower 4 bits are used to configure Timer 0 and the higher 4 bits are used to configure Timer 1

TMOD

Address → 89H

GATE1	C/T ₁	T1/M1	T1/MD	GATE0	C/T ₀	T0/M1	T0/MD
-------	------------------	-------	-------	-------	------------------	-------	-------

GATE bit is used to operate the TIMER 0/1 with respect to INT 0/1 pin.

- GATE1 enables and disables Timer 1 by means of signal brought to INT1
 - = 0 - Timer 1 operates only if INT1 is set
 - = 1 - Timer 1 operates regardless of INT1 bit
- GATE0 enables and disables Timer 0 by means of signal brought to INT0
 - = 0 - Timer 0 operates only if the INT0 bit is set
 - = 1 - Timer 0 operates regardless of INT0 bit
- C/T1 selects pulse to be counted up by timer/counter 1.
 - = 0 - timer counter pulse brought to pin T1
 - = 1 - timer counter pulse from internal oscillator
- C/T0 selects pulse to be counted up by timer/counter 0.
 - = 0 - timer counter pulse brought to T0 pin
 - = 1 - timer counter from internal oscillator

- TI/M1, TI/M0 - bits used to select operational mode of timer 1.

TI/M1	TI/M0	mode	description
0	0	0	13 bit timer
0	1	1	16 bit timer
1	0	2	8 bit auto reload
1	1	3	split mode or 2-8 bit timer mode

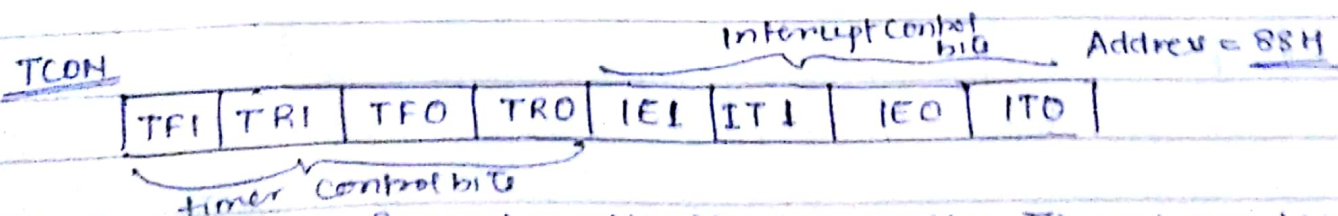
- TO/M1, TO/M0 bits used to select operational mode of timer.

TO/M1	TO/M0	mode	Description
0	0	0	13 bit
0	1	1	16 bit
1	0	2	8 bit autoreload
1	1	3	split mode 2 8-bit timer mode

(10) TCON (Timer Control)

It is one of the registers whose bits are directly in control of Timer operations.

Only 4 bits ~~are~~ of this register are used for control while rest of them is used for Interrupt control



- TFI bit is automatically set on the Timer1 overflow
- TR1 enables Timer 1
 - = 1 , Timer 1 is enabled
 - = 0 , Timer 1 is disabled

- TFO bit is automatically set on the Timer 0 overflow

- TRO bit enables Timer 0

= 1, Timer 0 is enabled

= 0, Timer 0 is disabled.

- IE1 (External Interrupt Edge flag) - set to 1 when a high to low edge signal is received on port 3.3 (INT1)

- IT1 (External Interrupt 1 signal type control bit)

Set to 1 by program to enable external interrupt 1 to be triggered by falling edge signal

- IE0 (External Interrupt 0 edge flag) - set to 1 when a high to low edge signal is received on port 3.2 (INT0).

- IT0 (External Interrupt 0 signal type control bit)

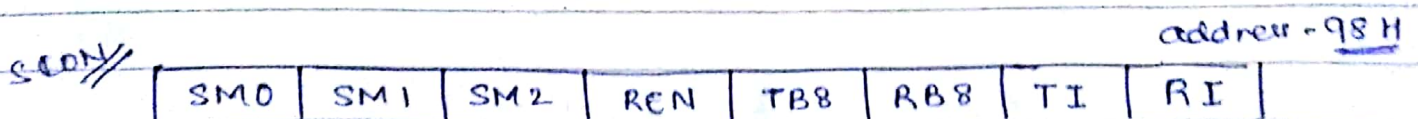
Set to 1 by program to enable External Interrupt triggered by falling edge.

② SCON (Serial Control register)

It is used to control the 8051's serial port. It is located at address 98 H.

Using SCON, you can control the operational modes of serial port.

SCON register also consists of bits that are automatically set when a byte of data is transmitted or received.



RI - (Receive Interrupt flag) is automatically set when one byte receive.

It signals that byte is received and should be read quickly prior to being replaced by new data.

TI - (Transmit Interrupt Flag) is automatically set when the last bit of one byte is send.

It is a signal to the processor that line is available for a new byte transmit.

RB8 - (Receives bit 8)

TB8 - (Transmitter bit 8)

set to transmit a logic 1 in the 9th bit

REN - (Reception enable bit) enables serial data reception

SM2 - Serial port mode 2 bit, also known as microprocessor communication enable bit,
when set, it enables communication in mode 2 or 3 and eventually in mode 1
when cleared it enables in mode 0.

SM1 - serial port mode bit 1
SM0 - serial port mode bit 0

} serial mode select

SM0	SM1	mode
0	0	0 (8 bit shift register)
0	1	1 (8 bit UART)
1	0	2 (9 bit UART)
1	1	3 (9 bit UART)

Port Operations

All 8051 microcontroller have 4 I/O ports each comprising 8 bits which can be configured as input or output. Hence, total 32 input/output pins allow the microcontroller to be connected with peripheral devices.

pin can be configured as 1 for input and 0 for output

port 0

characterised by two functions:-

- when external memory is used then the lower address byte (A0-A7) is applied on it, else all bits of this port are configured as input/output
- when P0 is configured as output then other ports consisting of pins with built-in pull-up register connected by its end to 5 volt supply.

port 1

It is a true I/O port as it doesn't have any alternative functions as in P0, but this port can be configured as general I/O only.

It has a built-in pull up register and is compatible with TTL circuit.

port 2

- similar to P0 when external Memory is used.

bits of this port occupy address intended for the external memory chip. This port can be used for higher address byte with address A8-A15.

port 3

In this port, functions are similar to other ports except that the logic 1 must be applied to appropriate bit of P3 register.

port of 8051 interrupt

(12) IE (Interrupt Enable)

It is used to enable or disable individual interrupts. If a bit is set, the corresponding interrupt is enabled and if the bit is cleared, the interrupt is disabled.

Address = A8_H

<u>IE</u>	EA	--	ET2	ES	ET1	EX1	ET0	EX0
-----------	----	----	-----	----	-----	-----	-----	-----

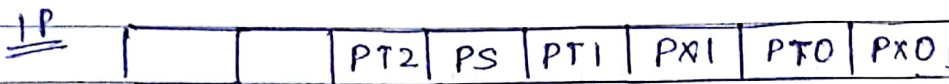
- EA - global interrupt enable/disable
= 0, disable all interrupt request
= 1, enables all individual interrupt request
- ES - enables/disables Serial interrupt
= 0, disable interrupt
= 1, enables interrupt
- ET1 - ~~Genable~~ enables/disables timer 1 interrupt
= 0, disable
= 1, enable
- EX1 - enables/disables external 1 interrupt
= 0, disable
= 1, enable

- ETO - enable/disable timer 0 interrupt
= 0, disable
= 1, enable.
- EXO - enable/disable external 0 interrupt.

⑬ IP (Interrupt priority) register

It is used to set priority of the interrupt as High or Low. If a bit is 0, the corresponding interrupt is assigned lower priority and if bit is set, the interrupt is assigned higher priority.

Address → B8 H



- PS - serial port interrupt priority bit
- PT1 - timer 1 interrupt priority
- PX1 - External 1 interrupt priority
- PT0 - timer 0
- PX0 - external 0

Imp. points

- if an ~~higher~~ interrupt of higher priority arrives while an interrupt is in progress, it will be immediately stopped and the higher priority interrupt will be executed
- if two interrupt request of diff. priority arrives at same time, higher one will service first
- if two interrupt request of equal priority arrive at the same time then, interrupt will be selected according to following list

1. External Interrupt INTO (PX0)
2. Timer 0 interrupt (PT0)
3. External Interrupt INT1 (PX1)
4. Timer 1 interrupt (PT1)
5. Serial Interrupt (PS)

8051 interrupts

— provides five sources of Interrupts.

in which

2 External Interrupts (INT0 & INT1)

3 Internal Interrupts (TFO, TFI & RI/TI)

— There are no software interrupts

— serial port interrupt (RI/TI) handled by SCON

— Timer1, Timer0 are handled by TCON

TCON, SCON, IE & IP control these interrupts.

External Interrupts can be edge triggered or level triggered.

These interrupts are processed internally by flags

⊙

Addressing Modes

It refers to how you are addressing a given memory location.

① Immediate Addressing mode

termed as immediate because 8-bit data is transferred immediately to the Accumulator.

ex MOV A, #6AH → Load 6AH data to accumulator

→ indicates that the operand is a data (8 bit)

In absence of #, hexadecimal no. would be taken as an address.

② Direct Addressing mode

here address of a data is given as an operand.

ex MOV A, 04H

→ the data stored in register 04H is moved to accumulator. As the 04H hold data 14H then 14H is moved to accumulator.

③ Register direct addressing mode

we use the register name directly as source operand.

ex MOV A, R4.

- move contents of Register R4 to the Accumulator

④ Register Indirect addressing mode

The address of data is stored in the register as operand

ex - MOV A, @R0

- moves the indirect RAM location specified by R0 to Accumulator.

Note only R0 & R1 are allowed to form a register indirect addressing Inst.

⑤ Indexed addressing

basically used for accessing data from look up table. here, the address of memory is indexed i.e. added to form actual address of memory.

Ex MOVC A, @A+DPTR

→ C - means code

content of register A is added with DPTR and the resultant is the address of memory location where data is copied to A register.

Instruction Set

① Arithmetic Instruction

Perform various basic operⁿ like addⁿ, subⁿ, division etc.

After execution, the result is stored in the first operand

ex ADDA, R1 → result of addition (A+R1) will be stored in Accumulator

(i) ADD dest, src

ADD A, Rn → Add the register to accumulator

ADD A, direct → add the direct byte to accumulator

ADDA, @Ri → add indirect RAM to the accumulator

ADDA, #data → add immediate data to A

(ii) ADDC (Add with carry)

ADDC A, Rn → add the register to accumulator with carry flag.

ADDC A, @Ri

ADDC A, #data

ADDC A, 25H

(iii) SUBB (Subtracts with borrow)

SUBB A, Rn

SUBB A, #data

SUBB A, @Ri

SUBB A, direct (Address)

(iv) SUB

SUB A, Rn

SUB A, @Ri

SUB A, #data

SUB A, direct

(v) INC (increment)

INC R_n → increments register by 1

INC A → increments Accumulator by 1

INC @R_i → increments ~~the~~ indirect ram by 1

INC DPTR → increment data pointer.

(vi) DEC (decrement)

DEC R_n, DEC A, DEC @R_i

(vii) MUL (multiplication)

MUL AB

(viii) Divide DIV

DIV AB divides A by B

(ix) DAA → decimal adjust of accumulator according to BCD code

② Data Transfer Instruction

(i) MOV dest, source

MOV A, R_n → move register to Accumulator

MOV A, @R_i → move indirect Ram to Accumulator

MOV A, #data → move immediate data to Accumulator

MOV R_n, A → moves accumulator to Register

MOV R_n, data → moves immediate data to Register

(ii) MOVX dest, source - used to copy from external RAM location to "accumulator"

Work with Accumulator Only.

(iii) MOVC dest, src → used to copy data from code or program memory into accumulator

~~MOV DPTR, #050H~~
MOV DPTR, #2500H
MOV A, #50H

MOVC A, @A+DPTR

(iv) Push direct → push direct byte onto the stack

(v) POP direct → pop the direct byte from stack

(vi) XCH → exchange

(vii) XCHD → exchange the bytes order

③ Logical Instruction

(i) ANL dest, src → AND operation; stores result in dest

(ii) ORL dest, src → OR operation

(iii) XRL dest, src → XOR operation

(iv) CLR A → Clears the Accumulator

(v) CPL A → Complements the Accumulator

(vi) SWAP A → swap nibbles within Accumulator

(vii) RL A → rotate left

(viii) RLC A → rotate left through carry

(ix) RR A → rotate right

(x) RRC A → rotate right through carry.

② Branching Instruction

Uncondⁿ jump - upon their execution a jump to a new address from where the program continues execution is executed.

Condⁿ jump - a jump to a new program location is executed only if a specified condⁿ is met.

① JMP

① Uncondⁿ JMP

- LJMP (Long Jump)
- SJMP (Short Jump) - {from -128 to +127 location}
- AJMP (Absolute Jump)

② Condⁿ JMP

- JC - (Jump if carry)
- JNZ - (Jump if not zero)
- JEZ - (Jump if equal)
- JNE - (Jump if not equal)
- CJNE (Compare and jump if not equal)

② CALL

① Uncondⁿ CALL

- ACALL (Absolute call)
- LCALL (Long call)

② Condⁿ call

Similar to condⁿ JMP like CNZ, CC etc.

③ RET → Return

④ RETI → (Returns from Interrupt Subroutine)